

# Security Review — PrivacyPortal & PrivacyPortalFactory

---

**Target:** `coti-io/coti-contracts @ commit 8a0c4928004dac7a6c8d50bde58a022b6912f963`  
**Method:** [pashov/skills](#) `solidity-auditor v3 ( c577eb7 )` — 12 parallel specialist agents (math-precision, access-control, economic-security, execution-trace, invariant, periphery, first-principles, asymmetry, boundary, numerical-gap, trust-gap, flow-gap) on `opus`, followed by orchestrator dedup, four-gate judging, and lead promotion. **Date:** 2026-09-05

---

## Scope

---

|                                     |                                                                                                                    |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Repository / commit</b>          | <code>coti-io/coti-contracts @ 8a0c4928004dac7a6c8d50bde58a022b6912f963</code>                                     |
| <b>Mode</b>                         | filename                                                                                                           |
| <b>Files reviewed</b>               | <code>contracts/pod/privacy/PrivacyPortal.sol</code> · <code>contracts/pod/privacy/PrivacyPortalFactory.sol</code> |
| <b>Confidence threshold (1-100)</b> | 80                                                                                                                 |

**Dedup / judging notes.** 25 unique (Contract, function) tuples in the raw agent output; all 25 covered before gating. Rejected at gating: `PrivacyPortal.rescueNative` (admin-only fee redirect, no unprivileged amplifier), `PrivacyPortalFactory._revokeRole` (cosmetic), `PrivacyPortalFeeLib.packFeeConfig` naming (out of scope). Dropped as cosmetic/admin-adversarial: the `requestWithdrawWithPermit` event-field mismatch and the `setLimits` "maxWithdraw = 0" freeze.

**Composite chain.** Chain: `[10]` + `[1]` at confidence 75 — a full-balance `rescueERC20` during remount feeds the stuck-withdrawal precondition; combined, the user loses the pTokens *and* the collateral leaves to `rescueRecipient`.

---

## Findings

---

[90] **1. Withdrawal whose pToken transfer succeeded but whose payout reverts is permanently stuck — both legs frozen, pTokens unburnable**

`PrivacyPortal._releaseWithdrawal` · Confidence: 90 · [agents: 9]

**Description** Once `pToken.requests(transferRequestId).status == Success`, a payout that reverts (a non-payable recipient on a native-wrapped portal, or an issuer-blacklisted recipient on an ERC-20 portal) makes `_releaseWithdrawal` revert forever while `cancelFailedWithdrawal` is unreachable (it requires `Failed / SystemFailed`), so the user's pTokens sit in portal custody outside `pendingBurnAmount` (never burnable) and the collateral is never released.

### Fix (Option A — allow-and-handle)

```
IWrappedNative(address(underlyingToken)).withdraw(withdrawal.amount);
(bool ok,) = payable(withdrawal.recipient).call{value: withdrawal.amount}("");
- if (!ok) {
-     revert EthTransferFailed();
- }
+ if (!ok) {
+     // do not revert: re-wrap and deliver the ERC-20, or credit a pull-payment claim
+     IWrappedNative(address(underlyingToken)).deposit{value: withdrawal.amount}();
+     underlyingToken.safeTransfer(withdrawal.recipient, withdrawal.amount);
+ }
```

### Fix (Option B — admin re-target / terminalize)

```
+ /// Admin-gated, whenPaused counterpart to adminRefundPendingDeposit for a
+ TransferPending
+ /// withdrawal whose transferRequestId is already Success: re-target
+ withdrawal.recipient,
+ /// or terminalize it while crediting pendingBurnAmount so custodied pTokens stay
+ burnable.
+ function adminResolveStuckWithdrawal(bytes32 withdrawalId, address newRecipient)
+     external onlyFactoryAdmin whenPaused nonReentrant { ... }
```

---

## [85] 2. Blacklist (and pause) are enforced only at request time; the collateral-release leg pays blacklisted recipients from a paused portal

PrivacyPortal.\_releaseWithdrawal · Confidence: 85 · [agents: 6]

**Description** `_releaseWithdrawal` — reachable permissionlessly via `triggerWithdrawalRelease` and via the attacker-forgeable `onPTokenTransferred` callback — checks neither the per-portal/factory blacklist nor `paused()`, so a withdrawal requested before a blacklisting or pause still pays out afterwards, defeating the control the code's own comment promises and letting collateral leave a paused portal during the `whenPaused` rescue/migration window.

### Fix (Option A — validate blacklist)

```
if (requestStatus != IPodERC20.RequestStatus.Success) {
    revert PTokenTransferNotSuccessful(withdrawal.transferRequestId,
requestStatus);
}
+ _checkNotBlacklistedAccount(withdrawal.recipient); // read the factory list via
```

```
bindingFactory
+ _checkNotBlacklistedAccount(withdrawal.user);
```

### Fix (Option B — validate blacklist + pause via bindingFactory)

```
+ if (paused() || _controllerFactory().withdrawalsPaused()) revert
WithdrawalsPaused();
+ if (blacklisted[withdrawal.recipient] ||
_controllerFactory().blacklisted(withdrawal.recipient)) {
+     revert AddressBlacklisted(withdrawal.recipient);
+ }
```

Either option must be paired with an admin path to re-target/refund a withdrawal that becomes unreleasable — otherwise it converts into finding #1.

---

## [85] 3. Remount rotates the pToken minter away, bricking the retired portal's adminRefundPendingDeposit for every still-Pending mint

PrivacyPortal.adminRefundPendingDeposit · Confidence: 85 · [agents: 9]

**Description** adminRefundPendingDeposit unconditionally calls pToken.invalidatePendingRequest (minter-gated by \_checkMinter) whenever the mint is Pending, but createPortalWithExistingPToken / setPTokenMinter move minter to the new portal while every escrow stays on the old one, so the refund reverts OnlyMinter(oldPortal) and the permissionless refundFailedDeposit cannot substitute (it needs SystemFailed), leaving depositor collateral unrefundable until an undocumented minter-swap or a requestKillMinAge -gated kill.

### Fix (Option A — authorize the owner/factory to invalidate)

```
// PodERC20.invalidatePendingRequest
- _checkMinter();
+ if (msg.sender != minter && msg.sender != owner()) revert OnlyMinter(msg.sender);
+ // plus a PrivacyPortalFactory forwarder (guarded by _requireFactoryOwnedPToken)
that
+ // adminRefundPendingDeposit falls back to when pToken.minter() != address(this)
```

### Fix (Option B — refuse remount while escrows are pending)

```
// PrivacyPortalFactory.createPortalWithExistingPToken, before setMinter
+ if (IPrivacyPortal(oldPortal).pendingEscrowCount() != 0) revert
OldPortalHasPendingEscrows(oldPortal);
```

### Fix (Option C — remove the kill delay before rotating)

```
+ IPodERC20(existingPToken).setRequestKillMinAge(0); // factory is the pToken's
Ownable owner
PodErc20Mintable(payable(existingPToken)).setMinter(portal);
```

---

#### [80] 4. Cross-factory remount skips every old-portal safety guard and rotates the minter away from a still-live portal

PrivacyPortalFactory.createPortalWithExistingPToken · Confidence: 80 · [agents: 4]

**Description** The `oldPortalNotPaused`, `native-mode`, and `retireDepositsForUpgrade` checks are nested under `if (oldPortal != address(0))` where `oldPortal` is read from this factory's own `portalForPToken`, so on the documented `transferPTokenOwnership` → `remount-on-new-factory` path they are all skipped while `setMinter(newPortal)` still runs — leaving the previous portal unpaused, un-retired, still serving withdrawals against its own collateral, with one pToken supply floating over two disjoint pools.

#### Fix

```
+ address prevMinter = PodErc20Mintable(payable(existingPToken)).minter();
+ if (prevMinter != address(0) && prevMinter != portal && prevMinter.code.length !=
0) {
+     if (!IPrivacyPortal(prevMinter).paused()) revert
OldPortalNotPaused(prevMinter);
+     // apply the same nativeWrappedUnderlying equality check against prevMinter
+ }
PodErc20Mintable(payable(existingPToken)).setMinter(portal);
```

#### [80] 5. createPortal returns a live, deposit-enabled portal before the one-way COTI registration has landed

PrivacyPortalFactory.createPortal · Confidence: 80 · [agents: 3]

**Description** Unlike `createPortalWithExistingPToken` (which calls `pauseByFactory()`), `createPortal` never pauses the clone and `initialize` hardcodes `isDepositEnabled = true`, so deposits made before the fire-and-forget `registerToken` message executes on COTI hit `TokenNotRegistered` (a retryable revert that leaves the mint `Pending`) and lock collateral behind an admin-only, pause-gated refund.

#### Fix (Option A — start closed)

```
IPrivacyPortal(portal).initialize(underlying, pToken, decimals,
nativeWrappedUnderlying, address(this));
+ IPrivacyPortal(portal).pauseByFactory(); // admin unpauses after observing
TokenRegistered on COTI
```

#### Fix (Option B — safe default in initialize)

```
// PrivacyPortal.initialize
- isDepositEnabled = true;
+ isDepositEnabled = false;
```

## [80] 6. Minter rotation in the remount path inspects none of the retiring portal's in-flight escrows (factory side of #3)

`PrivacyPortalFactory.createPortalWithExistingPToken` · Confidence: 80 · [agents: 2]

**Description** `setMinter(portal)` is executed unconditionally in the remount path with no check of the retiring portal's Pending deposit escrows, which is precisely what revokes the authority finding #3 depends on — and the test mocks (`MockPodErc20MintableForPortal` lacks `invalidatePendingRequest`; `MockPodERC20ForPortal` 's has no minter gate) cannot observe it.

### Fix (Option A — keep the retired portal authorized)

```
+ // let the pToken's Ownable owner (the factory) also authorize
invalidatePendingRequest,
+ // and add a factory forwarder so a retired portal's admin refund can terminalize
the mint
```

### Fix (Option B — drain before rotating)

```
+ // require the old portal to report zero Pending escrows (or invalidate them via
a
+ // factory-mediated call) before
PodErc20Mintable(existingPToken).setMinter(portal)
```

---

## [75] 7. Operator role can freeze all withdrawals via an unbounded `fixedFee` floor

`PrivacyPortal.setWithdrawFee` (and `PrivacyPortalFactory.setDefaultWithdrawFee`) · Confidence: 75 · [agents: 2]

**Description** `packFeeConfig` caps `percentageBps` at 10% but bounds `fixedFee` only by `uint96`, so an `OPERATOR_ROLE` holder — documented for "routine fee-parameter updates" — can set a withdraw-fee floor no wallet can pay (per portal, or factory-wide in one call) and block every `requestWithdrawWithPermit` with no `Paused` event, `paused()==false`, and no rescue enablement, an emergency-stop the role model reserves for `DEFAULT_ADMIN_ROLE`.

---

## [75] 8. `refundFailedDeposit` rejects mint status `Failed`, which the protocol's own ops tools write

`PrivacyPortal.refundFailedDeposit` · Confidence: 75 · [agents: 4]

**Description** The permissionless refund accepts only `SystemFailed`, but `killPTokenStaleRequest / invalidatePendingRequest` terminalize a stuck mint as `Failed`, so clearing a stale mint kills the permissionless refund and forces a full-portal pause plus admin refund for a single depositor — unlike `cancelFailedWithdrawal`, which accepts both terminal states (and `DepositEscrowStatus.Failed` is never written anywhere, leaving that branch dead).

---

[75] **9. Deposit limits and the fee are checked on the requested `amount` , but the position is minted from the measured `received`**

`PrivacyPortal._deposit` (and `depositNative`) · Confidence: 75 · [agents: 8]

**Description** `_checkDepositLimits(amount)` runs before the balance-delta measurement while escrow and mint use `received` , so with a fee-on-transfer underlying (an explicitly supported case, with a 5% mock in the repo) a deposit that passes `minDepositAmount` creates a position below `minWithdrawAmount` that can never be withdrawn, and the percentage fee is charged on the pre-fee notional.

---

[75] **10. `rescueERC20` has no reserve for withdrawals already in `TransferPending` , so the documented migration drains committed collateral**

`PrivacyPortal.rescueERC20` · Confidence: 75 · [agents: 4]

**Description** The portal keeps no aggregate of in-flight withdrawals and `rescueERC20` excludes only the pToken, so the pause → remount → rescue-full-balance procedure (exactly what the repo's tests perform) removes collateral that pending withdrawals still need; when those transfers settle `Success` the release reverts on balance, the pTokens are custodied but uncounted, and the user ends with neither leg.

---

[75] **11. Fee floor silently collapses to `fixedFee` (legally 0) when the oracle returns a zero rate**

`PrivacyPortal._portalFeeFloor` · Confidence: 75 · [agents: 4]

**Description** `resolvePortalFee` returns `(fixedFee, false)` on any zero rate and `_portalFeeFloor` discards the `usedDynamicPricing` flag, so a portal whose underlying was never priced (`createPortal` sets no peg), a cleared peg, or a stale-to-zero adapter degrades a configured percentage fee to the flat fee — which `packFeeConfig` allows to be 0 — with no revert and no event.

---

[75] **12. `setLimits` accepts `maxWithdraw < 2·minWithdraw` , leaving balances in the gap unredeemable**

`PrivacyPortal.setLimits` · Confidence: 75 · [agents: 1]

**Description** Each min/max pair is validated only against itself, so with `maxWithdraw < 2·minWithdraw - 1` a balance in `(maxWithdraw, 2·minWithdraw)` cannot be split into any legal sequence of withdrawals and its sub-minimum residue is permanently locked whenever deposits are disabled or the portal is retired.

---

[70] **13. `wrap` charges an execution-time fee with no caller-supplied bound**

PrivacyPortal.wrap · Confidence: 70 · [agents: 5]

**Description** wrap is the only value entry point that derives portalFee from the live floor instead of validating a caller-declared value, so a fee or oracle move between quote and inclusion (market drift, or an operator setDepositFee front-run) is silently absorbed as protocol fee and starves the forwarded mint budget, where deposit would have reverted with InsufficientPortalFee.

---

#### [70] 14. Escrow, burn, and withdrawal state are keyed on inbox request ids that are unique only per inbox instance

PrivacyPortal.\_deposit · Confidence: 70 · [agents: 7]

**Description** depositEscrows[requestId] and burnInFlight[burnRequestId] are written unconditionally and PodERC20.\_setRequestStatus does not guard writes of Pending, so if the admin re-points the pToken at a fresh inbox (configurePToken) whose per-target nonce restarts, colliding ids overwrite live escrows (destroying refund claims), double-count burnInFlightTotal (wedging batch burns), and can resurrect a stranded withdrawal's transferRequestId to Success — letting triggerWithdrawalRelease pay out collateral for a pToken transfer that never happened.

---

## Findings List

---

| #  | Confidence | Title                                                                        |
|----|------------|------------------------------------------------------------------------------|
| 1  | [90]       | Stuck withdrawal after successful pToken transfer — both legs frozen         |
| 2  | [85]       | Blacklist/pause not enforced on the collateral-release leg                   |
| 3  | [85]       | Remount minter rotation bricks adminRefundPendingDeposit                     |
| 4  | [80]       | Cross-factory remount skips all old-portal guards                            |
| 5  | [80]       | createPortal returns a live portal before COTI registration                  |
| 6  | [80]       | Remount rotates minter with no in-flight-escrow check (factory side of #3)   |
| 7  | [75]       | Operator can freeze withdrawals via unbounded fixedFee                       |
| 8  | [75]       | refundFailedDeposit rejects the Failed state ops tools write                 |
| 9  | [75]       | Deposit limits/fee on requested amount vs measured received                  |
| 10 | [75]       | rescueERC20 drains collateral committed to pending withdrawals               |
| 11 | [75]       | Fee floor collapses to fixedFee /0 on zero oracle rate                       |
| 12 | [75]       | setLimits allows an unpartitionable withdraw window                          |
| 13 | [70]       | wrap has no caller-side fee bound                                            |
| 14 | [70]       | Request-id reuse across inbox rotation corrupts escrow/burn/withdrawal state |

---

## Leads

---

*Vulnerability trails with concrete code smells where the full exploit path could not be completed in one analysis pass. These are not false positives — they are high-signal leads for manual review. Not scored.*

- **Permissionless release entry inherits the release-leg gap** —  
`PrivacyPortal.triggerWithdrawalRelease` — Code smells: no access control, no pause/blacklist check — the surface through which #2 fires and through which a resurrected `transferRequestId` (#14) would be cashed; the pause-bypass is asserted as intended by a remount test, which is what creates the #10 migration race.
- **burnInFlight overwrite double-counts burnInFlightTotal** —  
`PrivacyPortal.burnAccumulatedPTokens` — Code smells: `burnInFlight[id] = amount` (assignment) beside `burnInFlightTotal += amount` (increment) with no occupancy check — on an id collision (#14) the total inflates permanently and `available` underflows, wedging batch burns.
- **Forgeable OnlyPToken guard** — `PrivacyPortal.onPTokenTransferred` — Code smells: `transferAndCall` lets any holder make the pToken issue arbitrary calldata with `msg.sender == pToken` — harmless today only because the handler re-validates everything; any future pToken-trusting handler on the portal would be directly exploitable.
- **Unguarded oracle call on every user path, including exit** —  
`PrivacyPortal._portalFeeFloor` / `requestWithdrawWithPermit` — Code smells: `oracle.getLivePrices` with no try/catch; `IPodPriceOracle` only *promises* non-revert and `setPriceOracle` checks only code length — a reverting or gas-bombing adapter bricks deposits and new withdrawals factory-wide; no shipped adapter reproduces it.
- **Force- Failed transfer may close a withdrawal whose COTI leg settled** —  
`PrivacyPortal.cancelFailedWithdrawal` — Code smells: `killStaleRequest` / `invalidatePendingRequest` write `Failed` with no knowledge of COTI settlement — if the leg did settle, pTokens are custodied uncounted and the ticket closes with no collateral released; unverified whether COTI settlement can outlive a killed request.
- **pendingBurnAmount can be permanently over-stated** —  
`PrivacyPortal.finalizeBatchBurn` — Code smells: only `Success` decrements it; a burn that settled on COTI but lost its callback, then admin-killed to `Failed`, takes the failure branch with no corrective setter.
- **Outbound legs are unmeasured; refunds don't unwrap** —  
`PrivacyPortal._releaseWithdrawal` / `refundFailedDeposit` — Code smells: `unwrap` has no native-delta check (a shortfall would be paid from the fee balance); outbound fee-on-transfer direction untested (the mock's "portal never calls `transfer`" comment is false); `depositNative` users are refunded in WETH.



- **18-decimal assumption for the wrapped native** — `PrivacyPortal.depositNative` — Code smells: `amount` used both as wei and as token units for limits/fee-notional; the factory never asserts `decimals == 18` for the native-wrapped case; no deployment target found where it breaks.
- **Per-portal risk controls reset on remount** — `PrivacyPortal.initialize` — Code smells: the new clone starts with an empty per-portal blacklist, default limits, cleared fee overrides; only factory-wide controls survive; window exists only if the admin unpauses without reconfiguring.
- **One-shot, unacknowledged mother registration; underlying slot can be squatted** — `PrivacyPortalFactory.createPortal` — Code smells: fire-and-forget `sendOneWayMessage` with no error selector, `_requestMotherRegistration` private with no re-send, mappings never cleared, `createPortal` is the lowest-trust `DEPLOYER_ROLE` — a failed registration or a deployer pre-emption permanently occupies the underlying; unverified whether inbox retry re-drives the message.
- **Rotating the COTI peer on a live pToken strands in-flight requests** — `PrivacyPortalFactory.configurePToken` — Code smells: overwrites `trustedRemote`, so old-peer callbacks fail `UntrustedPeer` and requests never leave `Pending`; no paused precondition, unlike the remount path.
- **setPTokenMinter leaves the mapped portal advertising itself live** — `PrivacyPortalFactory.setPTokenMinter` — Code smells: no coupled retire/pause/mapping update; deposits revert `OnlyMinter` while `isDepositEnabled` / `paused()` still read open.
- **Constructor skips the code-existence checks the setters enforce** — `PrivacyPortalFactory.constructor` — Code smells: implementations/oracle only zero-checked at deploy; a codeless `portalImplementation` yields silently "initialized" clones that permanently occupy `portalForUnderlying` (no delete path), a codeless oracle bricks every portal's fee path.

⚠ This review was performed by an AI assistant. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. Team security reviews, bug bounty programs, and on-chain monitoring are strongly recommended. For a consultation regarding your projects' security, visit <https://www.pashov.com>